

XORception CTF Write-up

Author: @elesquina

Challenge Overview

XORception was a cryptography challenge that presented participants with a suspicious binary file intercepted from communication between members of a hacker group. The challenge description indicated that the file contained a secret message obfuscated using a simple XOR operation applied to an image file. Participants were provided with a 2.2MB binary file named `challenge.bin` and tasked with recovering the hidden message.

Initial Reconnaissance

Initial examination of the provided file revealed a 2,262,114-byte binary blob that did not match any standard file format signatures. Running the `file` command confirmed it was unrecognized data rather than a known file type. Hexadecimal inspection of the first 32 bytes showed `c4647a006449573e3447694e047c701569434b0134476ac74532344769aa6087`, the sequence which appeared to be random data with no immediately recognizable patterns.

The challenge description's mention of "simple XOR operation" and "image" suggested that a common image file format had been encrypted with XOR using a repeating key. Standard image formats like PNG, JPEG, GIF, and BMP all have distinctive magic byte signatures at the beginning of the file. The goal was to identify the XOR key by leveraging knowledge of these expected header bytes.

XOR Cryptanalysis Approach

XOR encryption with a repeating key is vulnerable to known-plaintext attacks. When the plaintext is known or can be reasonably guessed, the key can be recovered by XORing the ciphertext with the known plaintext. For image files, the magic bytes at the beginning serve as excellent known plaintext since they follow standardized specifications.

The standard PNG file format begins with an eight-byte signature: `89 50 4E 47 0D 0A 1A 0A`. This signature is followed by the first chunk, which is always the IHDR (Image Header) chunk. The IHDR chunk structure consists of a four-byte length field (always `00 00 00 0D` or 13 bytes for IHDR), followed by the four-byte chunk type identifier `49 48 44 52` (ASCII "IHDR").

Single-Byte Key Testing

The first approach involved testing all 256 possible single-byte XOR keys. For each potential key value from 0x00 to 0xFF, the first eight bytes of the encrypted file were XORed with the key and compared against the expected PNG signature. This brute-force approach would immediately reveal if a simple single-byte key had been used:

```
for key in range(256):
    xored = bytes([b ^ key for b in data[:8]])
    if xored == b'\x89PNG\r\n\x1a\n':
        print(f"Key found: {key}")
```

This test returned no matches, indicating that the encryption used either a multi-byte repeating key or a different image format. Testing was extended to cover JPEG signatures (`FF D8 FF`), GIF signatures (`47 49 46`), and BMP signatures (`42 4D`), but none produced valid results with single-byte keys.

Multi-Byte Key Analysis

With single-byte keys eliminated, attention turned to multi-byte repeating keys. The approach involved calculating what the key would need to be if the file were indeed a PNG, by XORing the encrypted bytes with the expected PNG header bytes. This calculation revealed potential key patterns:

```
png_signature = bytes.fromhex('89504e470d0a1a0a')
key_bytes = bytes([data[i] ^ png_signature[i] for i in range(8)])
# Result: 4d34344769434d34 = "M44GiCM4"
```

This eight-byte sequence "M44GiCM4" appeared promising as it consisted entirely of printable ASCII characters, which is common for deliberately chosen encryption keys. However, initial testing showed that simply repeating this eight-byte pattern did not produce a valid PNG file. The header signature matched correctly, but subsequent chunk data was corrupted.

Key Length Determination

Further analysis revealed the true nature of the key structure. By examining multiple known points in a PNG file structure, including the IHDR chunk type at bytes 12-15, it became apparent that the key pattern had a different repeat length than initially assumed. Calculating the key required to produce "IHDR" at the correct position showed:

```
expected_ihdr = b'IHDR'
key_at_12 = bytes([data[12+i] ^ expected_ihdr[i] for i in range(4)])
# Result: 4d343447 = "M44G"
```

Comparing this with the earlier eight-byte calculation revealed that the key actually repeated every six bytes, not eight. The pattern "M44GiC" repeated throughout the file. Testing with this six-byte key successfully produced valid PNG header bytes:

```
key = b'M44GiC'
decrypted = bytes([data[i] ^ key[i % 6] for i in range(len(data))])
# Signature: 89504e470d0a1a0a ✓
# Chunk length: 0000000d = 13 ✓
# Chunk type: IHDR ✓
```

Image Validation

After decrypting the entire file using the six-byte repeating key "M44GiC", the resulting PNG file was validated using Python's PIL (Pillow) library. The image successfully opened and revealed dimensions of 1589×900 pixels in RGBA color mode, confirming that the decryption was successful:

```
from PIL import Image
img = Image.open('flag.png')
print(f"Image: {img.size}, mode: {img.mode}")
# Output: Image: (1589, 900), mode: RGBA
```

The recovered image contained the challenge flag displayed visually as text rendered within the image. Visual inspection of the decrypted PNG revealed the flag embedded in the image content.

Key Findings

The XORception challenge demonstrated several fundamental cryptographic concepts. First, XOR encryption with repeating keys is vulnerable to known-plaintext attacks, especially when the plaintext follows predictable formats like standard file headers. Second, the key length can be determined through careful analysis of multiple known plaintext positions within the encrypted data.

The challenge also highlighted the importance of understanding file format specifications. Knowledge of PNG structure, including the fixed signature bytes and IHDR chunk format, was essential for recovering the encryption key. Without this domain knowledge, solving the challenge would have required significantly more brute-force computation.

Vulnerability Analysis

From a cryptographic perspective, the XOR cipher with a repeating key (also known as a Vigenère cipher when applied to text) suffers from fundamental weaknesses. The primary vulnerability stems from the fact that XORing the same key byte with different plaintext bytes allows statistical analysis and known-plaintext attacks. When the plaintext has known structure, as with file format headers, the key can be trivially recovered.

Modern encryption systems avoid these weaknesses by using keys that are at least as long as the plaintext (one-time pads), or by employing complex algorithms that provide confusion and diffusion properties (AES, ChaCha20). The repeating-key XOR cipher provides neither adequate key length nor algorithmic complexity to resist basic cryptanalysis.

Solution Script

The complete solution for the XORception challenge can be summarized in the following Python script:

```
#!/usr/bin/env python3

# Read the encrypted file
with open('challenge.bin', 'rb') as f:
    ciphertext = f.read()

# Known plaintext: PNG signature and IHDR chunk structure
png_signature = bytes.fromhex('89504e470d0a1a0a')
ihdr_length = bytes.fromhex('0000000d')
ihdr_type = b'IHDR'

# Calculate the XOR key (6 bytes: "M44GiC")
key = b'M44GiC'

# Decrypt the entire file
plaintext = bytes([ciphertext[i] ^ key[i % len(key)]
                    for i in range(len(ciphertext))])
```

```
# Write the recovered PNG
with open('flag.png', 'wb') as f:
    f.write(plaintext)

print("Flag recovered in flag.png")
```

Recovered Flag Image

After successfully decrypting the file using the XOR key "M44GiC", the recovered PNG image revealed the challenge flag:



The image contains the flag visually embedded within the PNG, confirming the successful completion of the XORception challenge.